Playbook: Suspicious PowerShell Flags (EQL)

Objective: Detect and validate obfuscated or encoded PowerShell executions using Elastic Security's EQL-based detection rules aligned with MITRE ATT&CK; Technique T1059.001.

1.0 Environment Overview

Component	Description
Platform	Elastic Cloud 9.2.0 (Security solution)
Integrations	Windows, Elastic Defend
Host System	Windows 11 Pro x64 test workstation
Agent Policy	My First Agent Policy (includes Elastic Defend, Windows integrations)
Repository Path	C:\gm-interview-capstone

2.0 Detection Rule Definition

Rule Name: Suspicious PowerShell Flags (EQL) Type: Event Correlation (EQL) Index Patterns: logs-*, endpoint.events.*, winlogbeat-*

EQL Query:
process where event.type in ("start", "process_started") and process.name : ("powershell.exe", "pwsh.exe") and (process.command_line : "*-enc*" or process.command_line : "*-nop*" or process.command_line : "*-NoProfile*" or process.command_line : "*-EncodedCommand*" or process.command_line : "*-ExecutionPolicy*Bypass*" or process.command_line : "*FromBase64String*" or process.command_line : "*iex (*")

3.0 Validation & Testing

Process telemetry was verified after enabling Elastic Defend. Command-line arguments were captured after enabling process creation auditing via auditpol. Alerts were confirmed using kibana.alert.rule.name filter in Security → Alerts.

4.0 Results

Detection successfully identified PowerShell executions containing encoded or obfuscation flags. Alert latency under 2 minutes per rule cycle. Full visibility achieved after telemetry correction.

5.0 Lessons Learned

1. Telemetry requires Elastic Defend or audit policy for Event ID 4688. 2. Correct correlation field is kibana.alert.rule.name. 3. Expanded index patterns prevent data omission. 4. Data-driven validation mirrors professional SOC practices.

6.0 Evidence and Artifacts

All artifacts are stored under C:\gm-interview-capstone\artifacts\figures\

#	Description	Filename
01	Workspace created	01_workspace_created.png
02	Elastic Cloud deployment	02_elastic_cloud_deployment.png
03	Fleet enrollment token	03_fleet_enrollment_token.png
04	Windows integration added	04_windows_integration_added.png
05	Agent healthy	05_agent_healthy_initial.png
06	Rule definition (EQL)	06_rule_definition_eql.png
07	Rule schedule/MITRE mapping	07_rule_schedule_mitre.png
08	Telemetry gap (no results)	08_discover_no_results_telemetry_gap.png
09	Elastic Defend added	09_elastic_defend_added.png
10	Rule updated with expanded indices	10_rule_index_patterns_updated.png
11	Rule preview no hits	11_rule_preview_no_hits.png
12	Manual rule run	12_rule_manual_run.png
13	Agent re-enrolled healthy	13_agent_reenrolled_healthy.png
14	Discover verified PowerShell	14_discover_verified_powershell.png
15	Discover command-line flags	15_discover_cmdline_flags.png
16	Rule manual run success	16_rule_manual_run_success.png
17	Alerts filtered by rule	17_alerts_filtered_by_rule.png
18	Alert details expanded	18_alert_details_panel.png

7.0 Conclusion

This playbook demonstrates complete detection engineering workflow using Elastic Security. The EQL rule effectively detects encoded or obfuscated PowerShell executions and validates end-to-end telemetry, alerting, and operational tuning.